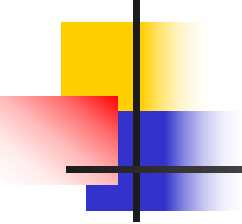


第5章 循环神经网络及应用



湖北大学 计算机学院
王雷春



目 录

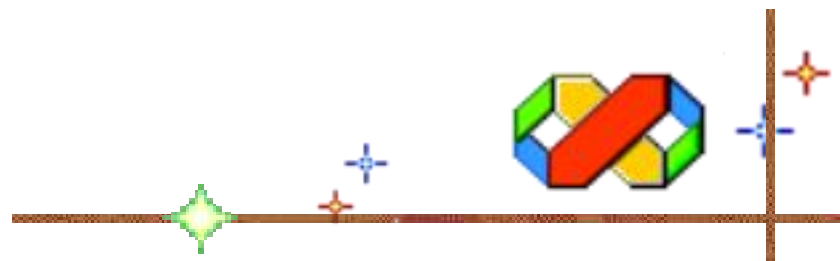
5.1 概述

5.2 RNN分类

5.3 标准RNN

5.4 经典RNN及使用方法

5.5 RNN应用及案例



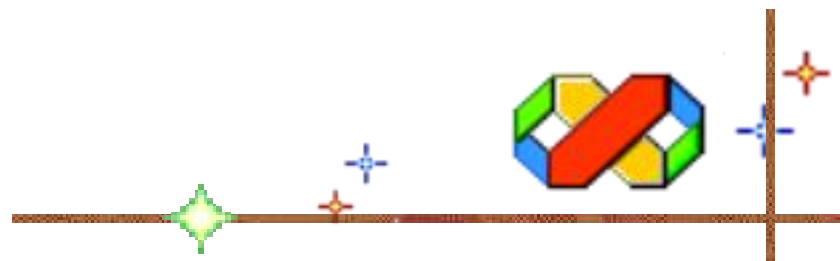


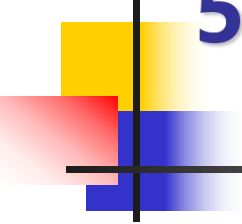
5.1 概述

5.1.1 背景与动机

5.1.2 什么是RNN

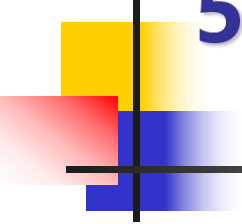
5.1.3 核心思想





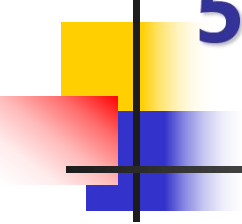
5.1.1 背景与动机

- ◆ (1)前馈神经网络的固有局限。标准的前馈神经网络（如全连接网络、卷积网络等）假设每个输入之间相互独立，无法处理像自然语言、语音、时间序列等真实场景中上下文紧密相关的数据。
- ◆ (2)变长序列处理难题。前馈神经网络的输入维度是固定的，无法直接处理长度可变的句子、音频或传感器数据。
- ◆ (3)对“记忆能力”的迫切需求。早期统计模型（如n-gram）只考虑了短距离历史，但存在数据稀疏问题，且难以捕捉长距离依赖，需要一种既能记忆历史信息、又能端到端训练的模型结构。



5.1.2 什么是RNN

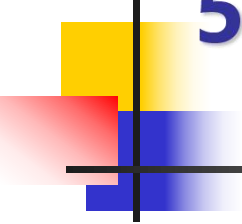
- ◆ 循环神经网络（RNN）是一种能够处理和建模序列数据的神经网络，通过循环结构使网络能够记忆和利用先前输入的信息，广泛用于语音、文本、时间序列等变长且上下文关联的任务。
- ◆ 与传统的前馈神经网络不同，RNN 采用循环连接结构，即当前时刻的输出不仅依赖于当前输入，还依赖于前一时刻的隐藏状态，从而能够对动态序列中的前后关联进行建模。



5.1.2 什么是RNN

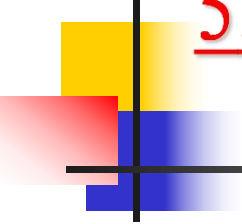
形象化比喻：读书

- ◆ 当读一个句子，如“苹果是我最喜欢的水果”。眼睛看到当前这个字或词“喜欢”（当前输入），回忆大脑里记忆的之前读过的字或词“苹果”、“是”、“我”等（隐藏状态），将二者结合（循环连接），然后准确理解“喜欢”的主体是谁，对象是什么。接着更新记忆，再去读下一个字。
- ◆ RNN的工作原理就是每个时间步把“当前输入”和“上一时刻记忆”混合，产生新的记忆和输出。



5.1.3 核心思想

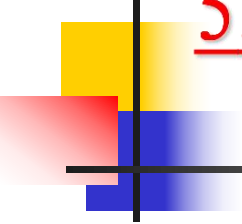
- ◆ (1)循环结构：网络在每个时间步将当前输入与上一时刻的隐藏状态共同作为计算依据，形成信息在网络内部持续传递的回路。
- ◆ (2)权重共享：所有时间步使用相同的权重矩阵，极大减少了参数量，并保证了模型对序列长度的适应性。
- ◆ (3)隐藏状态：一个随时间更新的记忆单元，它编码了从序列起始到当前时刻的所有历史信息。



5.2 RNN分类

1. 按结构类型

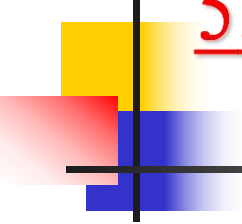
- ◆ (1)标准RNN。最简单的循环网络，隐藏层状态通过循环连接传递，计算简单，但存在梯度消失/爆炸问题，仅适合短序列建模。
- ◆ (2)长短期记忆网络（LSTM）：引入输入门、遗忘门、输出门和记忆单元，能有效捕捉长期依赖，是应用最广的变体。
- ◆ (3)门控循环单元（GRU）：LSTM的简化版，将遗忘门和输入门合并为更新门，参数更少、训练更快，性能与LSTM相近。
- ◆ (4)双向RNN（BiRNN）：同时包含正向和反向两个RNN层，能利用过去和未来的上下文信息，适用于文本分类、命名实体识别等需要完整上下文的任务。



5.2 RNN分类

1. 按结构类型

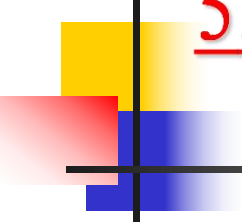
- ◆ (5)深度RNN (Deep RNN) : 将多个RNN层堆叠起来, 增强表达能力, 用于复杂模式的学习, 如语音识别、视频分析。



5.2 RNN分类

2. 按输入/输出类型

- ◆ (1)一对多 (One-to-Many) : 单个输入 $\rightarrow$ 序列输出, 本质是从单一向量生成序列, 即编码器只接收一次输入然后循环生成输出, 如文本生成 (起始词, 输出后续文字)、图像描述 (输入一张图像, 输出图像描述) 和音乐生成 (输入风格标签, 输出音符序列) 等。
- ◆ (2)多对一 (Many-to-One) : 序列输入 $\rightarrow$ 单个输出, 本质是将序列压缩为固定长度向量, 即只使用最后一个隐藏状态或对全部隐藏状态进行池化 (平均/最大), 如文本分类、情感分析、垃圾邮件检测等。



5.2 RNN分类

2. 按输入/输出类型

- ◆ (3)等长多对多 (Many-to-Many) : 输入输出序列等长, 每个时间步都有输出, 本质是序列到序列的对齐预测, 如词性标注、命名实体识别、机器翻译和语音帧分类等。
- ◆ (4)不等长多对多 (Many-to-Many) : 输入输出序列长度不同 (编码器-解码器结构), 本质是序列转换, 长度可变, 如机器翻译、文本摘要、对话系统、语音识别和代码生成等。



5.3 传统RNN

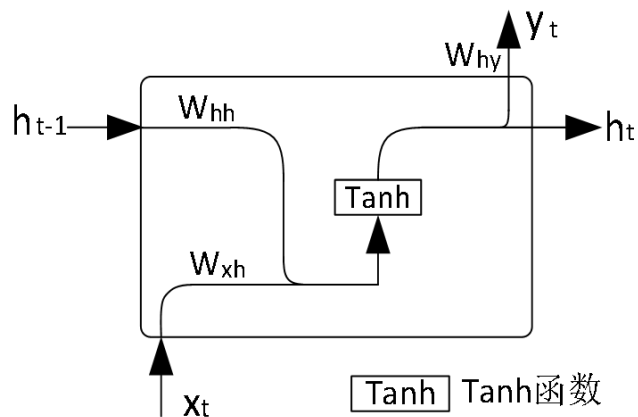
5.3.1 模型结构

5.3.2 工作流程

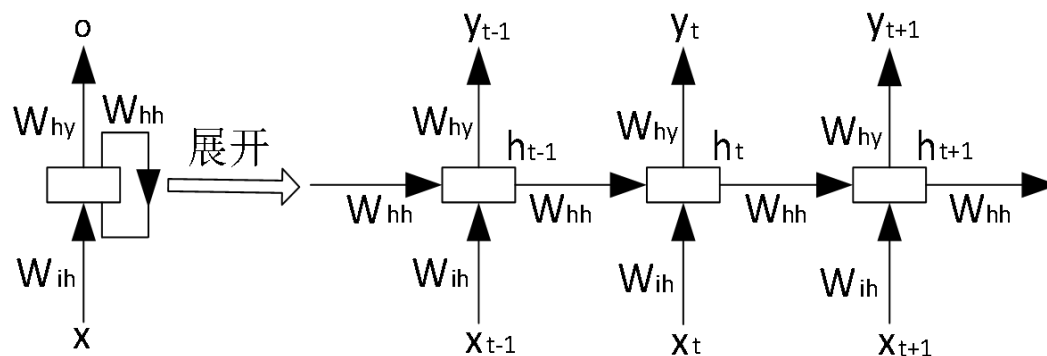
5.3.3 应用案例



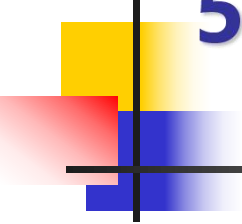
5.3.1 模型结构



(a) 单个RNN神经元



(b) RNN神经元序列



5.3.1 模型结构

x_t : t 时刻的输入（如时间序列、文本序列）；

h_t : t 时刻隐藏层的状态，即“记忆”。

y_t : t 时刻的输出；

W_{ih} : 输入层到隐藏层的权重矩阵，即 x_t 对应的权重矩阵；

W_{hh} : 隐藏层到隐藏层的权重矩阵，即 h_t 对应的权重矩阵；

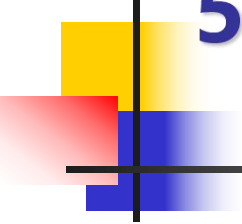
W_{hy} : 输出对应的权重矩阵；

h_t 的计算方法如下：

$$h_t = \tanh(W_{ih}x_t + W_{hh}h_{t-1} + b_h)$$

y_t 的计算方法如下

$$y_t = W_{hy}h_t + b_y$$



5.3.2 工作流程

1. 初始化

设定隐藏状态的初始值 h_0 ，通常初始化为全零向量或小随机数。

所有时间步共享同一组权重参数： W_{ih} 、 W_{hh} 、 W_{hy} ，以及偏置 b_h 和 b_y 。

2. 按时间步循环处理

对于长度为 T 的输入序列 $(x_1, x_2, \dots, x_n)$ ，RNN 依次执行：

时间步 t 的步骤：

(1) 读取当前输入 x_t 和上一时刻的隐藏状态 h_{t-1} ；

(2) 计算当前隐藏状态 h_t ：

$$h_t = \tanh(W_{ih}x_t + W_{hh}h_{t-1} + b_h)$$

这一步将新输入 x_t 与历史记忆 h_{t-1} 融合，经过非线性函数 ($\tanh$) 激活得到新的记忆。

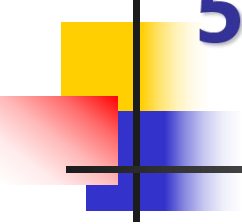
(3) 计算当前输出 y_t ：

$$y_t = W_{hy}h_t + b_y$$

输出可以是每个时间步都产生（如序列标注），也可以是只在最后一个时间步产生（如分类）。

(4) 将 h_t 传递到下一个时间步。

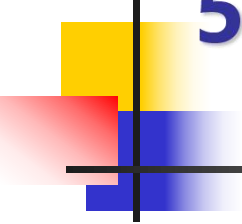
重复上述过程直到结束。



5.3.3 应用案例

【例5-1】使用传统RNN生成前 100 个斐波那契数。

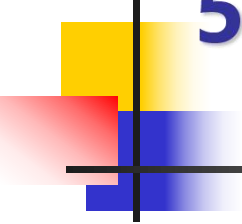
- ◆ 分析：
- ◆ (1)数据生成
- ◆ 斐波那契数列的递推关系 $F(n)=F(n-1)+F(n-2)$ 仅依赖前两项；
- ◆ 生成前100个斐波那契数 (0,1,1,2,3,5,8,...) 。
- ◆ (2)样本构造：使用长度为 2 的滑动窗口，每个样本(x_t, x_{t+1})对应目标 x_{t+2} 。RNN 的输入形状为(batch, seq_len=2, input_size=1)。



5.3.3 应用案例

【例5-1】使用传统RNN生成前 100 个斐波那契数。

- ◆ (3)模型设计
 - ◆ 单层 nn.RNN，隐藏单元数 32，batch_first=True。
 - ◆ 取最后一个时间步的隐藏状态，通过全连接层输出预测值。
 - ◆ 训练：MSE 损失，Adam 优化器，训练 500 轮，批量大小 16。
 - ◆ 预测：给定前两个数，模型输出预测值，四舍五入取整。



5.3.3 应用案例

【例5-1】使用传统RNN生成前 100 个斐波那契数。

程序代码

- ◆ 参见源代码文件

运行结果

- ◆ 斐波那契数列前 10 项: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
- ◆ --- 泛化测试 (未出现在训练集中) ---
- ◆ 输入 (13, 21) → 预测: 34, 真实: 34
- ◆ 输入 (21, 34) → 预测: 55, 真实: 55
- ◆ 输入 (34, 55) → 预测: 60, 真实: 89